Tokenization and Input Representations

pig7selene

September 5, 2026

Abstract

Tokenization defines the discrete alphabet through which a language model receives text and emits predictions. This chapter develops that interface from a string-to-token map through subword vocabulary construction and embedding lookup. It emphasizes the consequences of tokenization for sequence length, parameter allocation, reproducibility, and the engineering contracts that connect data preparation to model execution.

1 Introduction

Text reaches a Foundation Model through a deliberately narrow interface. The model does not receive characters, words, or bytes as linguistic objects; it receives a finite sequence of integer identifiers. That reduction is easy to overlook because tokenization is usually hidden behind a library call. It nevertheless fixes the model's input alphabet, determines how much of a context window a document occupies, and allocates a substantial parameter table. A tokenizer is therefore part of the model specification, not merely preprocessing.

The immediate goal of this chapter is to make the sequence entering a decoder-only Transformer mathematically explicit. Its longer-term purpose is practical: training systems count compute in tokens, form batches from token sequences, and reuse the same identifiers in the output softmax. The discussion treats tokenization as a stable contract between corpus processing, model training, evaluation, and generation.

2 From Text to Discrete Sequences

Let $\mathcal{S}$ denote the set of text strings after a chosen character encoding, and let $\mathcal{V}$ be a finite vocabulary. A tokenizer is a deterministic map from an input string to a finite sequence of vocabulary elements:

$$\tau : \mathcal{S} \to \mathcal{V}^*. \tag{1}$$

The star denotes the set of all finite sequences. In an implementation, each vocabulary element is assigned an integer identifier, so the output of τ is more conveniently written as

$$x = (x_1, \ldots, x_T), \quad x_t \in \{0, \ldots, |\mathcal{V}| - 1\}. \tag{2}$$

Here T is the tokenized length of the particular input, not a property of the vocabulary. Two strings with similar character counts can have very different values of T, and the computational cost of a Transformer is governed primarily by token positions rather than character positions.

> **Definition 1: Tokenizer.** A tokenizer consists of a finite vocabulary $\mathcal{V}$, a convention for converting text into an initial sequence of atomic symbols, a segmentation procedure that returns elements of $\mathcal{V}$, and an inverse decoding convention on the subset of sequences that it regards as valid.

The definition includes more than a merge list. Normalization, whitespace handling, the choice between Unicode characters and bytes, and the treatment of reserved symbols all affect the map in Equation 1. A tokenizer need not be reversible over arbitrary input strings: a normalizer may intentionally collapse distinctions. It should, however, make its contract explicit. In a training system, the same contract must be used for corpus preparation, training examples, evaluation prompts, and generation-time decoding. Otherwise the model is trained and evaluated on different discrete languages.

The vocabulary is usually much smaller than the space of possible strings. A word-level vocabulary makes each frequent word inexpensive in sequence length, but it either assigns rare forms to an unknown symbol or grows without bound. A character-level vocabulary avoids most out-of-vocabulary failures, but represents common words by long sequences. Subword tokenization occupies the useful middle ground: common fragments may become single tokens, while uncommon strings are composed from smaller pieces. Sennrich, Haddow, and Birch adapted BPE to this open-vocabulary setting and showed that subword units can replace a fixed-vocabulary back-off mechanism in neural translation [1].

3 Vocabulary Design and Reserved Symbols

A vocabulary contains ordinary text pieces and a small set of symbols with control semantics. Typical examples include a beginning-of-sequence marker, an end-of-sequence marker, padding, and reserved delimiters for message roles or document boundaries. These are token IDs whose embeddings are learned in exactly the same way as ordinary token embeddings, but whose positions carry a convention established by the training data and runtime.

The crucial design rule is that a special token must be handled as an atomic unit before ordinary segmentation. If a chat delimiter is sometimes decomposed into text pieces and sometimes inserted as one reserved ID, the model sees two incompatible representations of the same structural event. Conversely, a token that is reserved but never appears in training has no learned semantics merely because it has a vocabulary entry. Tokenizer configuration and data formatting must therefore be versioned together.

Vocabulary size also creates a direct parameter trade-off. A larger vocabulary can shorten sequences by storing more frequent substrings, but it enlarges both the input embedding table and, in many decoder-only models, the output classifier. It also changes the empirical distribution of targets: a very common token receives many updates, while a rare token receives few. There is no vocabulary size that is independently optimal; the appropriate choice depends on corpus coverage, languages, model width, and the intended context length.

4 Byte Pair Encoding

BPE originated as a compression algorithm that repeatedly replaces frequent adjacent symbol pairs with newly introduced symbols [2]. Its appeal for neural text processing is not compression alone. The same procedure constructs a vocabulary that can represent frequent strings compactly while retaining a base alphabet for rare strings. Modern BPE tokenizers differ in details, but the merge construction is sufficiently simple to derive directly.

Consider a corpus represented as sequences over an initial alphabet $\mathcal{V}_0$. The alphabet may be characters, bytes, or a reversible encoding of bytes. At merge step m, let $c_{m(a,b)}$ count occurrences of the adjacent pair (a, b) in the current representation. BPE selects a most frequent pair,

$$(a_m, b_m) = \arg\max_{a,b} c_{m(a,b)}, \tag{3}$$

introduces a new symbol u_m denoting the concatenation of a_m and b_m, and replaces the selected pair according to a fixed overlap convention. After M merges, the learned vocabulary is the base vocabulary together with the introduced symbols:

$$\mathcal{V}_M = \mathcal{V}_0 \cup \{u_1, ..., u_M\}. \tag{4}$$

The recurrence exposes the central inductive bias. BPE does not discover linguistic morphemes by definition; it merges pairs that are frequent under its current representation. A frequent substring can become one token even if it is not a word, while a morphologically meaningful but rare substring may remain split. This is often appropriate for language modeling, whose immediate objective is to assign probabilities to symbol sequences, but it prevents us from treating token boundaries as a ground-truth linguistic analysis.

Algorithm 1: BPE vocabulary training. Start from an initial corpus representation over $\mathcal{V}_0$. At each step, count adjacent pairs, choose a pair with maximum count under a deterministic tie rule, introduce its concatenated symbol, and replace its eligible occurrences throughout the corpus. Record the ordered merge. Stop after the prescribed number of merges or when no pair is eligible.

Training the merge vocabulary and encoding a new string are separate procedures. Training estimates which pairs deserve vocabulary entries from a corpus. Encoding begins from the base symbols of a new string and applies only merges recorded during training, in their learned priority order. Implementations commonly store a rank for each merge and repeatedly select the best available adjacent pair. The rank order, base-symbol construction, and pre-tokenization policy are all part of the tokenizer artifact; changing any of them changes the IDs in Equation 2.

Byte-level BPE makes the base alphabet cover bytes rather than an application-specific set of characters. Provided the byte encoding itself is reversible, every input byte sequence can then be represented without an unknown token; rare text simply falls back to more base symbols. GPT-2 used a byte-level version of BPE, illustrating how this choice supplies a fixed vocabulary while retaining coverage of arbitrary text bytes [3]. By contrast, SentencePiece emphasizes training subword models directly from raw sentences, avoiding a language-specific requirement for pre-tokenized word boundaries [4]. These choices are implementation conventions, not interchangeable defaults: they influence whitespace behavior, multilingual coverage, and the reproducibility of a dataset pipeline.

5 Embedding Lookup

The output of a tokenizer becomes useful to a neural network through an embedding table. Let d be the model width and let

$$E \in R^{|\mathcal{V}| \times d} \tag{5}$$

be a trainable matrix. The vector associated with token ID x_t is the corresponding row,

$$e_t = E[x_t] \in R^d. \tag{6}$$

This operation is a row lookup, although automatic-differentiation systems often implement it as multiplication by a one-hot vector. If q_t is the one-hot representation of x_t, then $e_t = q_t^T E$. The lookup view is operationally more useful: only the rows selected by a batch receive direct embedding gradients. A vocabulary item that appears rarely receives few such updates, while a control token inserted into every sequence may receive many.

For a batch of B padded sequences of maximum length T, the token-ID tensor has shape $B \times T$ and the embedding output has shape $B \times T \times d$. The embedding table contributes $|\mathcal{V}|d$ trainable scalar parameters before any positional representation or Transformer layer is considered. This count is elementary, yet it explains why tokenization choices cannot be separated from model budgeting.

6 Positional Input Representations

The embedding table associates a vector with a token identity, not with a location in a sequence. The word piece for a name has the same lookup vector whether it occurs first or last. A Transformer consequently needs an additional source of positional information. In the original Transformer formulation, position-dependent vectors are added to token embeddings before the stacked attention and feed-forward layers [5]. We write the initial state at position t as

$$h_t^0 = e_t + p_t, \tag{7}$$

where $p_t \in R^d$ is a positional representation. It may be a learned absolute embedding, a deterministic vector, or part of a later attention computation. The present point is narrower than a comparison of position-encoding schemes: tokenization fixes the positions to which any such scheme is applied. Altering a tokenizer can therefore change both the length and the positional geometry of every training example, even when the underlying text is unchanged.

Some implementations share the input embedding matrix with the output projection used to score the next token. Weight tying reduces parameters and links the geometry of input and output symbols, but it is an architectural option rather than a tokenizer requirement. The original Transformer described shared embedding and pre-softmax weights in some settings [5].

7 Token Counts and Engineering Implications

For a corpus $\mathcal{D}$, define its token count under tokenizer τ by

$$N_{\tau(\mathcal{D})} = \sum_{s \in \mathcal{D}} |\tau(s)|. \tag{8}$$

This quantity recurs throughout Foundation Model engineering. Pretraining budgets are normally reported in tokens, batches are assembled in token positions, and the cost of processing a fixed document collection varies with $N_{\tau(\mathcal{D})}$. A tokenizer that splits a corpus into longer sequences consumes more context positions and can increase the attention work associated with a training example. It may nevertheless be preferable if its base representation improves coverage or avoids brittle text normalization. The correct comparison is therefore not a slogan such as "fewer tokens is better," but a joint assessment of coverage, modeling behavior, parameter cost, and systems cost.

The same caution applies to evaluation. Perplexity and average negative log-likelihood are defined per model token, so their numerical values are meaningful only relative to a specified tokenizer and normalization pipeline. Comparing token-level losses across incompatible tokenizations can confound changes in modeling quality with changes in the units being predicted.

8 Implementation Considerations

A minimal tokenizer implementation should expose four testable operations: encode text to IDs, decode valid IDs to text, identify reserved tokens, and serialize the complete vocabulary plus segmentation rules. Round-trip tests should be performed on representative text, including whitespace, non-ASCII characters, delimiters, and malformed inputs where the contract specifies behavior. The test does not prove that a tokenizer is linguistically desirable; it establishes that the discrete interface is stable.

For BPE, a small implementation is especially instructive. Begin with a corpus of short strings, represent each string as a sequence of base symbols, count adjacent pairs, apply the merge rule in Equation 3, and store the resulting ranks. Then encode held-out strings using those ranks. The exercise makes two facts concrete: merge training is corpus dependent, and inference-time segmentation is constrained by a fixed learned artifact. Both facts become important when training data, checkpoints, or evaluation sets are revised.

9 Summary

Tokenization is one of the first irreversible modeling decisions in a language-model pipeline. It defines the symbols that a model will embed, predict, cache, shard, and evaluate. A sound implementation makes its discrete mapping, special-token conventions, merge rules, and serialization format explicit. The resulting sequence of IDs is the input to the decoder-only computation graph developed in the accompanying Transformer Architecture chapter.

References

[1] R. Sennrich, B. Haddow, and A. Birch, "Neural Machine Translation of Rare Words with Subword Units," in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Association for Computational Linguistics, 2016, pp. 1715–1725. doi: 10.18653/v1/P16-1162.

[2] P. Gage, "A New Algorithm for Data Compression," *C Users Journal*, vol. 12, no. 2, pp. 23–38, 1994, [Online]. Available: https://www.derczynski.com/papers/archive/BPE_Gage.pdf

[3] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language Models are Unsupervised Multitask Learners," technical report, 2019. [Online]. Available: https://cdn.openai.com/better-language-models/language-models.pdf

[4] T. Kudo and J. Richardson, "SentencePiece: A Simple and Language Independent Subword Tokenizer and Detokenizer for Neural Text Processing," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, Association for Computational Linguistics, 2018, pp. 66–71. doi: 10.18653/v1/D18-2012.

[5] A. Vaswani *et al.*, "Attention Is All You Need," in *Advances in Neural Information Processing Systems 30*, 2017, pp. 5998–6008.